

Disjoint Set Union (DSU) and Kruskal's Algorithm

Complete Notes and Documentation

Table of Contents

1. Introduction to DSU
2. Find Operation
3. Union Operation
4. Path Compression
5. DSU Code
6. Introduction to Kruskal's Algorithm
7. Working of Kruskal's Algorithm (Dry Run)
8. Difference Between DSU Tree and MST Tree
9. Kruskal Code
10. Time and Space Complexity Analysis
11. Final Summary

1. Introduction to DSU

DSU (Disjoint Set Union), also called Union-Find, is a data structure used to maintain several separate groups (sets).

It supports two operations:

- **Find(x)**
- **Union(x,y)**

It is mainly used for:

- Cycle detection
- Kruskal's Algorithm
- Connected Components
- Dynamic Graph Problems

Initial State

Suppose we have 7 nodes:

1 2 3 4 5 6 7

Initially each node forms its own set:

{1} {2} {3} {4} {5} {6} {7}

Parent Array

Initially every node becomes its own parent. This means every node is the root of its own tree.

Node	Parent
1	1
2	2
3	3
4	4
5	5
6	6
7	7

2. Find Operation

Find(x) returns the representative (root) of x's set.

Example Scenario

Parent array state:

1 → 1

2 → 1

3 → 1

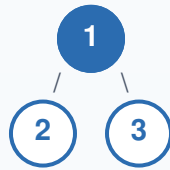
Execution:

Find(3) follows the parent pointers: 3 → 1.

Therefore: **Find(3) = 1**

Similarly: **Find(2) = 1**

TREE REPRESENTATION



3. Union Operation

$\text{Union}(x, y)$ merges two sets.

INITIAL INDEPENDENT SETS

Set A:



Set B:



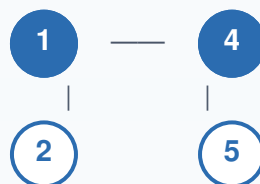
When executing $\text{Union}(2, 5)$:

1. First, find the roots of both elements:

- $\text{Find}(2) = 1$
- $\text{Find}(5) = 4$

2. We do not connect 2 and 5 directly. Instead, we connect the roots: **1 --- 4**

RESULTING TREE AFTER UNION(2,5)



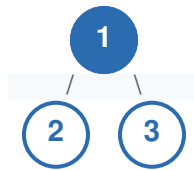
Important Rule

Union always connects roots, not the original nodes.

Doubt Solved

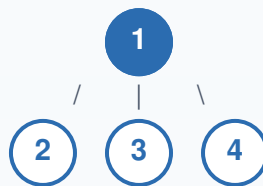
Question: If we write $\text{Union}(3, 4)$, will 4 become a child of 3?

Answer: No. Suppose we have the tree where node 3 belongs to root 1 ($\text{Find}(3) = 1$):



Therefore, $\text{Union}(3, 4)$ transforms into $\text{Union}(1, 4)$, and 4 gets connected directly to root 1.

RESULTING TREE STRUCTURE



Thus, DSU always joins roots.

4. Path Compression

Suppose a tree becomes highly skewed and linear:

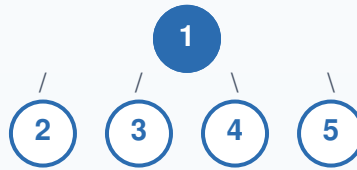
SKEWED TREE BEFORE PATH COMPRESSION



When executing $\text{Find}(5)$, the algorithm travels sequentially: $5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1$.

After **Path Compression**, every node visited along the path is re-attached directly to the root node:

FLATTENED TREE AFTER PATH COMPRESSION



Now, every future find operation becomes almost constant time.

5. DSU Code

Find Parent Function

```
int findParent(int node, vector& parent)
{
    if(node == parent[node])
        return node;

    return parent[node] = findParent(node, parent);
}
```

Union Function

```
void unionSet(int u, int v, vector& parent)
{
    int pu = findParent(u, parent);
    int pv = findParent(v, parent);

    parent[pv] = pu;
}
```

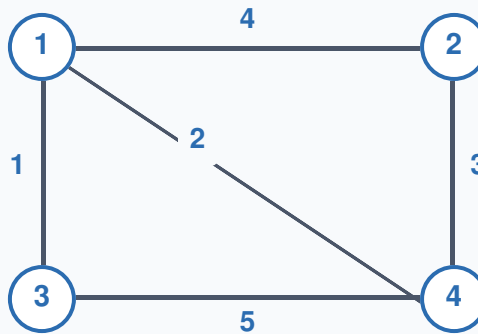
6. Introduction to Kruskal's Algorithm

Purpose: Find the Minimum Spanning Tree (MST) of a given connected, undirected graph.

Greedy Strategy: Always choose the minimum weight edge that does not create a cycle inside the current workspace.

Example Graph

INPUT GRAPH STRUCTURE



Edges List:

- $(1,3) = 1$
- $(1,4) = 2$
- $(2,4) = 3$
- $(1,2) = 4$
- $(3,4) = 5$

Step 1: Sort Edges

Sorted order based on weight:

1. $(1,3) = 1$
2. $(1,4) = 2$
3. $(2,4) = 3$
4. $(1,2) = 4$
5. $(3,4) = 5$

7. Working of Kruskal's Algorithm (Dry Run)

Initial Parent Array: $1 \rightarrow 1, 2 \rightarrow 2, 3 \rightarrow 3, 4 \rightarrow 4$

Initial Total MST Weight: 0

Iteration 1

- **Edge:** $(1,3) = 1$
- $\text{Find}(1) = 1, \text{Find}(3) = 3$
- Different roots $\rightarrow$ **Accept edge**

- $\text{Union}(1, 3) \rightarrow \text{Weight} = 1$
- **Parent State:** $1 \rightarrow 1, 2 \rightarrow 2, 3 \rightarrow 1, 4 \rightarrow 4$

Iteration 2

- **Edge:** $(1, 4) = 2$
- $\text{Find}(1) = 1, \text{Find}(4) = 4$
- Different roots $\rightarrow$ **Accept edge**
- $\text{Union}(1, 4) \rightarrow \text{Weight} = 1 + 2 = 3$
- **Parent State:** $1 \rightarrow 1, 2 \rightarrow 2, 3 \rightarrow 1, 4 \rightarrow 1$

Iteration 3

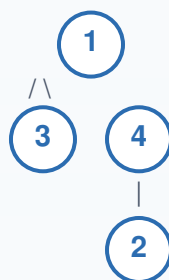
- **Edge:** $(2, 4) = 3$
- $\text{Find}(2) = 2, \text{Find}(4) = 1$
- Different roots $\rightarrow$ **Accept edge**
- $\text{Union}(2, 4) \rightarrow \text{Weight} = 3 + 3 = 6$
- **Parent State:** $1 \rightarrow 1, 2 \rightarrow 1, 3 \rightarrow 1, 4 \rightarrow 1$

Termination Condition

Since vertices $V = 4$, the required edges for the Minimum Spanning Tree is $V - 1 = 3$. The algorithm stops executing further iterations.

Final MST Layout

FINAL MINIMUM SPANNING TREE (MST)

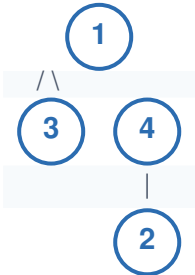
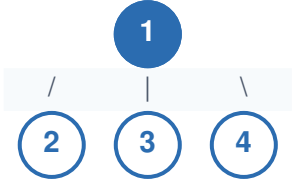


- **Edges selected:** $(1, 3) = 1, (1, 4) = 2, (2, 4) = 3$
- **Total Cost:** 6

8. Difference Between DSU Tree and MST Tree

Question: Why are the DSU tree and the MST tree different?

Answer: They represent completely different mathematical concepts:

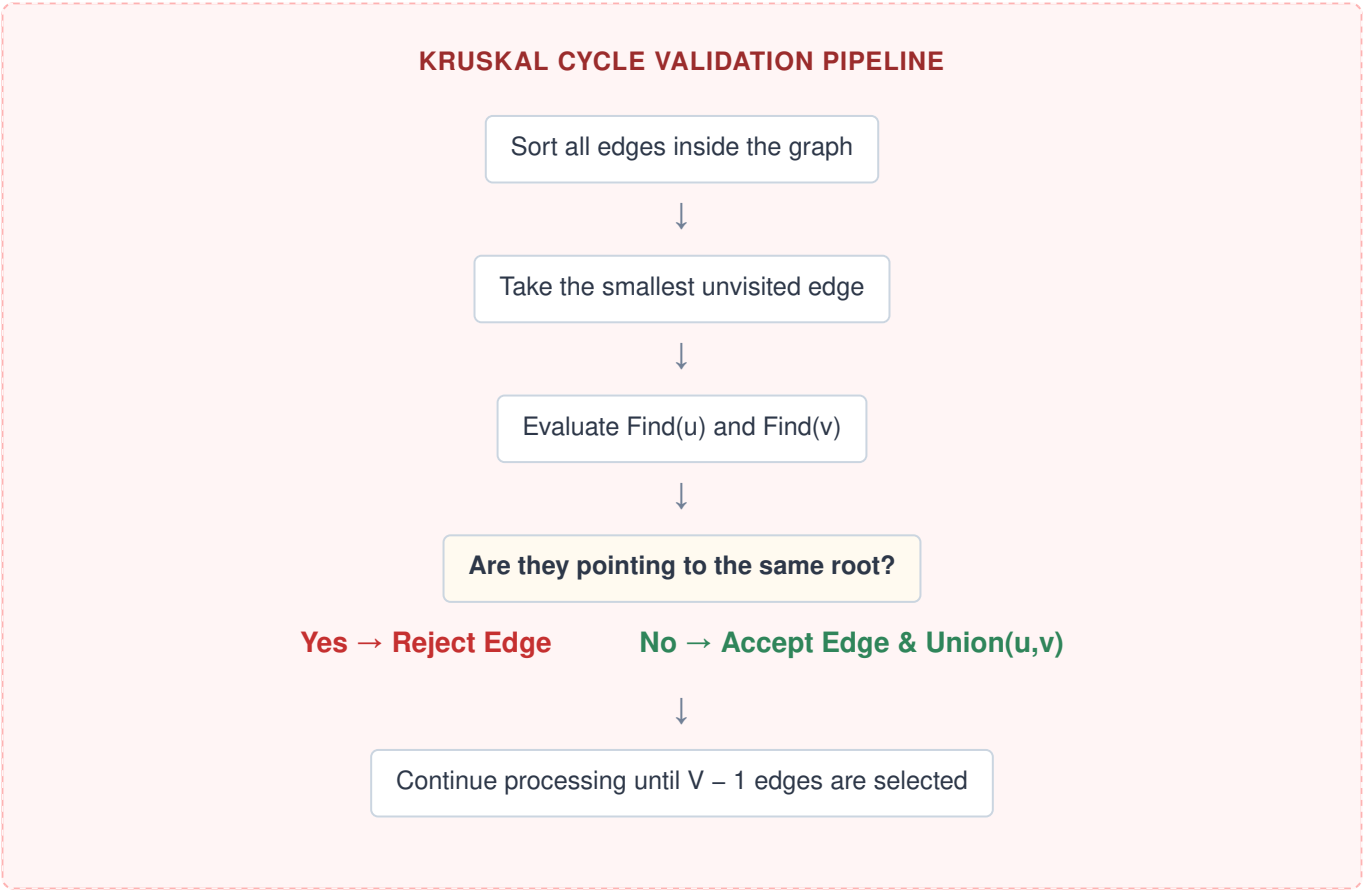
MST Tree	DSU Tree
Contains the actual minimum weight physical graph edges selected. 	Stores only logical parent relationships internal to the disjoint set tracking. 

DSU does not preserve graph edges. Its only purpose is to efficiently determine whether nodes belong to the same component or partition.

Cycle Detection Showcase

Suppose we evaluated the next edge: $(1, 2) = 4$

- $\text{Find}(1) = 1$
- $\text{Find}(2) = 1$
- **Same root detected!** Therefore, adding this edge would create a cycle. Hence, reject the edge.



| 9. Kruskal Code

```

class Solution {
public:

    int findParent(int node, vector& parent)
    {
        if(node == parent[node])
            return node;

        return parent[node] = findParent(parent[node], parent);
    }

    void unionSet(int u,int v,vector& parent)
    {
        int pu = findParent(u,parent);
        int pv = findParent(v,parent);

        parent[pv] = pu;
    }

    int spanningTree(int V, vector> adj[])
    {
        vector>> edges;

        for(int u=0;u<V;u++)
        {
            for(auto it : adj[u])
            {
                int v = it[0];
                int wt = it[1];

                edges.push_back({wt,{u,v}});
            }
        }

        sort(edges.begin(),edges.end());

        vector parent(V);

        for(int i=0;i<V;i++)
            parent[i]=i;

        int mstWeight=0;

        for(auto edge : edges)
        {
            int wt = edge.first;
            int u = edge.second.first;

```

```

        int v = edge.second.second;

        if(findParent(u,parent)!=findParent(v,parent))
        {
            mstWeight += wt;
            unionSet(u,v,parent);
        }
    }

    return mstWeight;
}
};

```

10. Time and Space Complexity Analysis

Let V be the number of vertices and E be the number of edges.

Time Complexity Breakdown

- **Sorting edges:** $O(E \log E)$
- **Parent initialization array:** $O(V)$
- **Processing edges:** For every edge, we perform $\text{Find}(u)$, $\text{Find}(v)$, and $\text{Union}(u, v)$. With path compression implemented, the find operation takes $O(\alpha(V))$ where $\alpha(V)$ represents the inverse Ackermann function.

Practically, $\alpha(V) < 5$, meaning processing edges works in $O(E \alpha(V)) \approx O(E)$.

Overall Time Complexity: $O(E \log E)$ (Dominant term)

Space Complexity Breakdown

- **Parent tracking array:** $O(V)$
- **Edge storage list:** $O(E)$
- **Overall Space Complexity:** $O(V + E)$

11. Final Summary

DSU Core Operations

- **Find(x):** Returns the unique root identifier of x.
- **Union(x,y):** Merges the components containing x and y by connecting their roots.
- **Path Compression:** Flattens tree hierarchies to provide near constant time performance.

Kruskal's Protocol Summary

1. Sort all edges by their raw weights in non-decreasing order.

2. Iterate and pick the smallest active edge.
3. Check roots inside DSU. Same root $\rightarrow$ reject. Different root $\rightarrow$ accept and execute Union.
4. Terminate execution after selecting exactly $V - 1$ edges.

Kruskal uses DSU for efficient cycle detection and stands as one of the most important greedy graph algorithms.